

cub3D - Support de soutenance

Version du document: etat du code au 7 mars 2026.

Depot analyse: 'cub3d/' dans ce repository.

Objectif: permettre a deux personnes de revoir rapidement tout le projet, de comprendre le flux d'execution complet, et de preparer une soutenance 42 sur le mandatory uniquement.

1. Vue d'ensemble du projet

'cub3D' est un mini moteur 3D en vue subjective base sur le raycasting. On ne dessine pas de vrais volumes 3D. On part d'une carte 2D composee de cases, puis on envoie un rayon pour chaque colonne de l'ecran. Chaque rayon touche un mur, et cette distance est convertie en hauteur de mur affichee sur l'ecran. C'est exactement l'idee historique utilisee par des jeux comme Wolfenstein 3D.

Dans ce projet, il y a quatre grands blocs:

1. le parsing du fichier '.cub'
2. l'initialisation du moteur et de MiniLibX
3. la boucle de jeu: input, mouvement, rendu
4. le nettoyage complet en cas de fermeture normale ou d'erreur

Le mandatory demande en particulier:

- MiniLibX
- une fenetre qui reste exploitable
- quatre textures de murs distinctes selon l'orientation 'N/S/E/W'
- deux couleurs differentes pour le sol et le plafond
- les fleches gauche/droite pour tourner
- 'W', 'A', 'S', 'D' pour se deplacer
- 'ESC' et la croix rouge pour quitter proprement
- un fichier '.cub' en argument
- une map valide, fermee, avec un seul joueur
- un message 'Error' suivi d'un message explicite en cas de probleme

Le code actuel respecte ce cadre sans activer de bonus.

2. Les structures de donnees importantes

Tout le projet repose sur quelques structures definies dans 'cub3d/includes/cub3d.h'.

2.1 't_color'

```
'typedef struct s_color'
```

Cette structure stocke trois entiers 'r', 'g', 'b'.

Elle sert pendant le parsing pour memoriser les couleurs du sol et du plafond avant de les convertir en entier '0xRRGGBB' utilisable au rendu.

2.2 't_map'

```
'typedef struct s_map'
```

Cette structure represente la carte 2D finale:

- 'grid': tableau de chaines, une ligne par ligne de map
- 'width': largeur maximale de la map
- 'height': nombre de lignes
- 'player_dir': orientation initiale 'N', 'S', 'E' ou 'W'

- 'player_x', 'player_y': position de depart dans la grille

C'est la structure logique du monde de jeu.

2.3 't_texture'

```
'typedef struct s_texture'
```

Cette structure contient les chemins texte vers les quatre fichiers XPM:

- 'north'
- 'south'
- 'west'
- 'east'

Attention: ici on ne stocke encore que des chemins, pas les images chargees en memoire.

2.4 't_cub'

```
'typedef struct s_cub'
```

C'est la structure de sortie du parsing. Elle contient:

- les chemins des textures dans 'tex'
- les couleurs du sol et du plafond
- la map finale

On peut la voir comme la description complete de la scene.

2.5 't_player'

```
'typedef struct s_player'
```

Cette structure represente l'etat runtime du joueur:

- 'x', 'y': position reelle en 'double', au centre des cases
- 'angle': angle de vue en radians

Le parsing donne un point de depart en grille. Ensuite 'setup_game()' convertit cette position en coordonnees reelles.

2.6 't_img'

```
'typedef struct s_img'
```

Cette structure encapsule une image MiniLibX:

- 'img': pointeur MLX brut
- 'addr': adresse du buffer pixel
- 'bpp': bits par pixel
- 'line_len': taille d'une ligne memoire
- 'endian'
- 'w', 'h'

Elle est utilisee pour:

- la frame complete a dessiner
- chaque texture murale chargee

2.7 't_ray'

```
'typedef struct s_ray'
```

C'est la structure de calcul d'un rayon pendant le raycasting. Elle contient tout ce qui sert a tracer une colonne:

- direction du rayon

- coordonnees de case courante
- distances DDA ('delta_x', 'delta_y', 'side_x', 'side_y')
- sens des pas ('step_x', 'step_y')
- type de face touchee ('side')
- distance perpendiculaire ('perp')
- hauteur de ligne a dessiner ('line_h')
- bornes ecran ('start', 'end')
- coordonnees de texture ('wall_x', 'tex_x', 'step', 'pos')

2.8 't_mlx'

```
'typedef struct s_mlx'
```

Cette structure regroupe les pointeurs MiniLibX:

- 'mlx': contexte global
- 'win': fenetre
- 'w', 'h': taille de la fenetre
- 'frame': image hors ecran que l'on remplit avant affichage

2.9 't_game'

```
'typedef struct s_game'
```

C'est la structure runtime principale. Elle regroupe tout:

- 'cub': la scene parsee
- 'mlx': le contexte graphique
- 'player': l'etat du joueur
- 'tex_no', 'tex_so', 'tex_we', 'tex_ea': textures deja chargees par MLX
- 'floor_color', 'ceil_color': couleurs compactees en entier
- 'keys[1024]': etat des actions de clavier

C'est la structure qui circule dans la boucle du jeu.

2.10 't_gm'

```
'typedef struct s_gm'
```

Cette structure est un contexte temporaire de parsing. Elle sert uniquement pendant la lecture du '.cub'.

Elle stocke:

- le descripteur de fichier
- la ligne en cours de lecture
- les flags indiquant si chaque config a ete vue
- les lignes de map en construction
- les dimensions en cours
- une version paddee de la map pour verifier la fermeture

2.11 't_flood'

```
'typedef struct s_flood'
```

Cette structure n'est pas utilisee dans la version actuelle. C'est vraisemblablement un reste d'une approche precedente de validation de map par flood fill. Elle n'a pas d'effet sur le runtime actuel.

3. Ordre d'execution complet du programme

C'est la partie la plus importante pour la soutenance. Quand l'evalateur demande "Que se passe-t-il du lancement jusqu'au premier affichage?", il faut pouvoir raconter exactement le

chemin d'appel.

3.1 'main()' dans 'src/main.c'

Le programme commence dans 'main(int ac, char **av)'.

Ordre reel:

1. verifier qu'il y a exactement un argument
2. initialiser toute la structure 't_game'
3. parser le fichier '.cub'
4. convertir les donnees parsees en donnees runtime
5. initialiser MiniLibX et charger les textures
6. installer les hooks clavier / fermeture
7. lancer la boucle MLX
8. nettoyer a la sortie

Dans le code:

- 'print_error("usage: ./cub3D <map.cub>")' si le nombre d'arguments est faux
- 'init_game_struct(&game)'
- 'parse_cub(&game.cub, av[1])'
- 'setup_game(&game)'
- 'mlx_setup(&game, 1024, 768, "cub3D")'
- 'mlx_hook(...)' pour les evenements
- 'mlx_loop_hook(...)' pour demander 'render_frame()' a chaque tour
- 'mlx_loop(...)' pour confier la main a MLX

3.2 'init_game_struct()'

Cette fonction appelle d'abord 'init_cub()' puis 'init_images()'.

But:

- mettre tous les pointeurs a 'NULL'
- mettre les dimensions a '0'
- remettre les couleurs a des valeurs neutres
- remettre toutes les touches a '0'

L'interet est simple: si une erreur arrive plus tard, le cleanup peut tester proprement ce qui a deja ete alloue.

3.3 'parse_cub()'

Cette fonction construit toute la scene.

Ordre interne:

1. 'init_cub(cub)'
2. 'check_path_map(path_map)'
3. 'init_gm(&gm, cub)'
4. 'open(path_map, O_RDONLY)'
5. 'read_cub_file(&gm)'
6. 'build_map(&gm)'
7. retourner '1' si tout est bon, sinon nettoyer via 'parse_fail()'

Le parsing est le bloc le plus important apres le raycasting. On le detaille dans le chapitre suivant.

3.4 'setup_game()'

Une fois la scene validee, on passe du monde "parsing" au monde "runtime".

Cette fonction:

- place le joueur au centre de sa case avec '+ 0.5'

- convertit 'N/S/E/W' en angle en radians avec 'dir_to_angle()'
- compacte 'RGB' du plafond et du sol en entier 24 bits

Exemple:

- si le joueur est sur la case 'x = 10', 'y = 5', la vraie position devient '10.5', '5.5'
- si le joueur regarde 'N', l'angle est ' $-\pi / 2$ '

3.5 'mlx_setup()'

Cette fonction realise deux choses:

1. creer le contexte graphique
2. charger les quatre textures murales

Ordre interne:

- 'setup_mlx_core()'
- 'load_texture()' pour 'north'
- 'load_texture()' pour 'south'
- 'load_texture()' pour 'west'
- 'load_texture()' pour 'east'

Si une seule de ces etapes echoue, on appelle 'mlx_destroy()' puis on retourne '0'.

3.6 Installation des hooks

Dans 'main()', trois hooks sont installes:

- evenement '2': appui touche -> 'key_press'
- evenement '3': relachement touche -> 'key_release'
- evenement '17': fermeture fenetre -> 'on_close'

Puis 'mlx_loop_hook()' demande a MLX d'appeler 'render_frame()' regulierement.

3.7 Boucle principale

Une fois dans 'mlx_loop()', le programme vit selon ce schema:

1. MLX detecte les evenements utilisateur
2. les hooks mettent a jour l'etat des touches
3. 'render_frame()' est appelee
4. 'render_frame()' met a jour le joueur
5. 'render_frame()' dessine plafond + sol
6. 'render_frame()' lance 'cast_rays()'
7. l'image calculee est envoyee dans la fenetre
8. le cycle recommence

4. Parsing complet du fichier '.cub'

Le parsing est volontairement strict. Il doit garantir que le moteur recupere une scene coherente.

4.1 'check_path_map()'

Cette fonction verifie deux points avant meme d'ouvrir le fichier:

- l'extension doit etre '.cub' via 'gm_has_cub_ext()'
- le chemin ne doit pas etre un dossier via 'not_directory()'

Si ce test echoue, on sort immediatement avec un message d'erreur explicite.

4.2 'init_gm()'

'init_gm()' prepare le contexte temporaire 't_gm':

- 'fd = -1'
- 'line = NULL'
- 'in_map = 0'
- flags des configs a '0'
- dimensions a '0'
- tableaux a 'NULL'

C'est l'equivalent parsing de 'init_game_struct()'.

4.3 'read_cub_file()'

Cette fonction lit le fichier ligne par ligne avec 'get_next_line()'.

Ordre:

1. 'gm->line = get_next_line(gm->fd)'
2. tant qu'on a une ligne:
 - appeler 'read_line(gm)'
 - si erreur: liberer la ligne courante, vider GNL, retourner '0'
 - sinon liberer la ligne et lire la suivante
3. a la fin, appeler 'gnl_clear()'
4. retourner '1'

Le point important: le buffer statique de 'get_next_line' est nettoye explicitement avec 'gnl_clear()' pour eviter qu'il reste de la memoire allouee entre deux parse.

4.4 'read_line()'

C'est la vraie logique de classification d'une ligne.

Ordre:

1. 'trim_eol()' supprime '
' et '
'
2. si on n'est pas encore dans la map et que la ligne est vide, on l'accepte
3. si on n'est pas encore dans la map et que la ligne ressemble a une ligne de map:
 - on verifie que toutes les configs ont deja ete lues avec 'has_all_config()'
 - on bascule 'gm->in_map = 1'
4. si on est dans la map:
 - la ligne doit etre une vraie ligne de map
 - on l'ajoute avec 'store_map_line()'
5. sinon, on attend une ligne de config:
 - 'gm_is_config_line()'
 - 'gm_parse_config()'

C'est cette fonction qui impose la regle du sujet: la map doit toujours etre la derniere partie du fichier.

4.5 'gm_is_config_line()'

Cette fonction ne parse pas encore. Elle verifie juste si la ligne commence par un identifiant valide:

- 'NO'
- 'SO'
- 'WE'
- 'EA'
- 'F'
- 'C'

Elle tolere les espaces avant l'identifiant.

4.6 'gm_parse_config()'

Cette fonction distribue vers le bon parseur.

Cas possibles:

- 'NO ...' -> 'set_texture(&north, ...)'
- 'SO ...' -> 'set_texture(&south, ...)'
- 'WE ...' -> 'set_texture(&west, ...)'
- 'EA ...' -> 'set_texture(&east, ...)'
- 'F ...' -> 'gm_set_color(&floor, ...)'
- 'C ...' -> 'gm_set_color(&ceiling, ...)'

4.7 'set_texture()' et la logique texture

'set_texture()':

1. appelle 'trim_token()' pour nettoyer les espaces autour du chemin
2. refuse les doublons via '*flag'
3. verifie que le chemin existe et n'est pas un dossier avec 'check_path()'
4. stocke le chemin alloue dans la structure 't_cub'

Important a expliquer en soutenance:

- a ce stade, on verifie surtout le chemin
- la validite réelle XPM sera confirmee plus tard par 'mlx_xpm_file_to_image()'

4.8 'gm_set_color()' et le parsing RGB

'gm_set_color()' delegue a 'parse_rgb()'.

'parse_rgb()' fait le travail suivant:

1. lire 'r' avec 'parse_value()'
2. verifier '0 <= r <= 255'
3. verifier une virgule
4. lire 'g'
5. verifier '0 <= g <= 255'
6. verifier une virgule
7. lire 'b'
8. verifier '0 <= b <= 255'
9. verifier qu'il ne reste rien d'illicite apres

Cela accepte par exemple:

- 'F 220,100,0'
- 'F 220, 100 , 0'

Et cela refuse:

- des valeurs negatives
- des valeurs > 255
- des composantes manquantes
- du texte parasite apres les nombres
- un doublon de 'F' ou 'C'

4.9 'store_map_line()' et 'gm_push_line()'

Quand une ligne de map est validee:

- 'gm_push_line()' l'ajoute au tableau dynamique 'gm->lines'
- 'store_map_line()' met a jour 'gm->width' si cette ligne est la plus longue

'gm_push_line()' appelle 'gm_expand()' si le tableau doit grandir. C'est un tableau dynamique classique qui double sa capacite.

4.10 'build_map()'

Après lecture complète du fichier, 'build_map()' valide la géométrie finale.

Ordre interne:

1. vérifier qu'on a bien lu une map et qu'elle a au moins une ligne
2. créer une version padding avec 'gm_pad_lines()'
3. vérifier qu'elle est fermée avec 'gm_is_closed()'
4. libérer la version padding temporaire
5. affecter la vraie map à 'cub->map' avec 'assign_map()'
6. vérifier qu'il y a exactement un joueur avec 'find_player()'

Si la map n'est pas fermée ou si le joueur n'est pas valide, la fonction retourne '0'.

4.11 Pourquoi 'gm_pad_lines()' est une bonne solution

Le sujet dit que les espaces sont une partie valide de la map.

Le problème est que des lignes peuvent avoir des longueurs différentes. Si on essaye de vérifier la fermeture directement sur les chaînes d'origine, il faut gérer une quantité de cas limites énorme.

La stratégie choisie ici est propre:

1. on crée une grille plus grande, remplie d'espaces
2. on copie la map au milieu, décalée d'une case
3. on teste chaque case jouable ('O', 'N', 'S', 'E', 'W')
4. si une de ses quatre cases voisines est un espace, alors la map est ouverte

Cette technique transforme tous les "trous" et les bords hors carte en espaces. C'est simple, robuste et très facile à expliquer à l'oral.

4.12 'find_player()'

Cette fonction parcourt toute la map:

- chaque fois qu'elle voit 'N', 'S', 'E' ou 'W', elle stocke la position et l'orientation
- elle incrémente un compteur
- elle retourne '1' seulement si le compteur final vaut exactement '1'

Donc:

- '0' joueur -> erreur
- '2' joueurs ou plus -> erreur

4.13 'parse_fail()'

C'est le garde-fou mémoire du parsing.

En cas d'erreur, il:

- ferme le fichier si nécessaire
- libère la ligne courante si nécessaire
- libère les lignes de map accumulées
- vide le buffer statique de GNL
- libère ce qui a déjà été stocké dans 't_cub'
- affiche 'Error' + message

Cette fonction est une raison importante pour laquelle le projet ne fuit pas en cas d'erreur de parsing.

5. get_next_line dans ce projet

Le projet embarque sa propre version de 'get_next_line' dans 'cub3d/get_next_line/'.

5.1 Idée générale

Le principe est simple:

- garder un buffer statique entre deux appels
- lire petit à petit le fichier
- s'arrêter dès qu'on a une ligne complète
- renvoyer cette ligne
- conserver le reste pour l'appel suivant

5.2 'get_next_line()'

Ordre:

1. récupérer l'adresse du buffer statique avec 'gnl_buffer()'
2. vérifier 'fd' et 'BUFFER_SIZE'
3. lire tant qu'il n'y a pas encore de '
' dans le buffer
4. si 'read()' a renvoyé '-1', nettoyer via 'free_if_err()'
5. si le buffer est vide, retourner 'NULL'
6. sinon extraire une ligne via 'extract_line()'

5.3 'read_it()'

Cette fonction lit un bloc depuis le fichier:

- alloue un 'tmp'
- appelle 'read()'
- si le buffer principal est vide, crée une chaîne vide
- concatène l'ancien buffer et le bloc lu
- remplace le buffer principal par cette nouvelle chaîne

5.4 'extract_line()'

Si un '
' est présent:

- couper la chaîne au bon endroit
- renvoyer la partie de gauche
- garder la partie de droite dans le buffer statique

Sinon:

- renvoyer tout le buffer
- remettre le buffer à 'NULL'

5.5 'gnl_clear()'

Cette fonction est capitale dans ce projet. Elle vide explicitement le buffer statique. Comme le parsing peut quitter tout sur erreur, il fallait une fonction de reset pour éviter toute fuite entre deux parse.

6. Initialisation graphique et chargement des textures

Tout ce bloc est dans 'src/mlx_init.c'.

6.1 'setup_mlx_core()'

Cette fonction crée le cœur MLX:

1. stocker largeur et hauteur
2. 'mlx_init()'
3. 'mlx_new_window()'
4. 'mlx_new_image()' pour creer le framebuffer hors ecran
5. 'mlx_get_data_addr()' pour recuperer l'adresse memoire du framebuffer

L'interet du framebuffer hors ecran est simple: on dessine toute l'image dans un buffer, puis on l'envoie d'un coup a la fenetre. Cela evite le scintillement et simplifie le rendu.

6.2 'load_texture()'

Cette fonction charge une texture XPM:

- 'mlx_xpm_file_to_image()' charge le fichier et donne sa taille
- 'mlx_get_data_addr()' donne l'adresse brute des pixels

Si l'un des deux echoue, la fonction affiche une erreur explicite.

6.3 'mlx_setup()'

'mlx_setup()' orchestre tout:

- initialiser MLX
- charger 'north'
- charger 'south'
- charger 'west'
- charger 'east'

Cette fonction garantit que, quand elle retourne '1', le moteur a tout ce qu'il faut pour dessiner.

6.4 'mlx_destroy()'

Cette fonction fait le nettoyage symetrique:

- detruire la frame
- detruire les textures
- detruire la fenetre
- detruire l'affichage MLX
- remettre tous les pointeurs a 'NULL'

Ce reset a 'NULL' est une bonne pratique defensive: si 'mlx_destroy()' est appelee dans plusieurs chemins, on evite les doubles destructions.

7. Input et deplacement

7.1 Ce que demande exactement le sujet

Le sujet demande:

- fleche gauche et fleche droite pour regarder
- 'W', 'A', 'S', 'D' pour se deplacer
- 'ESC' pour quitter

Le code actuel suit exactement cette exigence.

7.2 'map_key()'

Cette fonction convertit un 'keycode' brut recu de MLX en action interne.

Correspondances actuelles:

- fleche gauche -> 'ACT_TURN_LEFT'

- fleche droite -> 'ACT_TURN_RIGHT'
- 'W' -> 'ACT_MOVE_FWD'
- 'A' -> 'ACT_MOVE_LEFT'
- 'S' -> 'ACT_MOVE_BACK'
- 'D' -> 'ACT_MOVE_RIGHT'

L'avantage de cette normalisation est important:

- 'render.c' ne depend plus de keycodes bruts
- le code de mouvement manipule des intentions ('aller en avant', 'tourner a droite') et non des codes clavier

7.3 'key_press()' et 'key_release()'

Ces deux fonctions ne deplacent pas directement le joueur.

Elles font une chose simple:

- marquer une action comme active ('1') quand on appuie
- marquer cette action comme inactive ('0') quand on relache

C'est une bonne architecture car elle permet des mouvements fluides:

- maintenir 'W' pour avancer en continu
- maintenir fleche droite pour tourner en continu
- combiner 'W' + 'D' ou 'W' + fleche droite

7.4 'on_close()'

Cette fonction est appelee sur 'ESC' ou sur la croix rouge.

Elle:

- detruit MLX
- libere 't_cub'
- appelle 'exit(0)'

C'est la sortie propre du programme.

8. La boucle de rendu

Tout se passe dans 'src/render.c'.

8.1 'render_frame()'

A chaque frame:

1. 'update_player(g)'
2. 'draw_floor_ceiling(g)'
3. 'cast_rays(g)'
4. 'mlx_put_image_to_window(...)'

C'est le coeur du moteur.

8.2 'update_player()'

Cette fonction lit l'etat des actions dans 'keys[]':

- avant/arriere
- strafe gauche/droite
- rotation gauche/droite

Puis elle:

- met a jour l'angle du joueur
- appelle 'move_player()'

Les vitesses sont fixees par:

- 'MOVE_SPEED 0.03'
- 'ROT_SPEED 0.04'

8.3'move_player()'

Cette fonction calcule une position candidate.

Pour l'avance/recul:

- 'nx = x + cos(angle) * fwd'
- 'ny = y + sin(angle) * fwd'

Pour le strafe:

- angle lateral = 'angle + PI / 2'
- on ajoute une composante laterale sur 'nx' et 'ny'

Ensuite, on ne valide pas d'un coup la position finale. On teste separerement:

- l'axe X
- l'axe Y

C'est une bonne technique pour glisser le long des murs au lieu de rester bloque brutalement sur les diagonales.

8.4'can_move()'

Cette fonction teste quatre coins autour du joueur avec un petit rayon 'COLLIDER_R = 0.20'.

On ne teste pas seulement le centre du joueur. Sinon il pourrait entrer partiellement dans un mur. Le petit collider donne une collision plus realiste.

Comme 'map_is_wall()' considere aussi les espaces comme des murs, on n'a pas le droit d'avancer hors de la zone ferme de la map.

8.5'draw_floor_ceiling()'

Cette fonction remplit le framebuffer:

- moitie haute -> couleur plafond
- moitie basse -> couleur sol

Ce n'est pas du bonus de floor casting texture. Dans le mandatory du sujet, le sol et le plafond sont des couleurs, pas des textures.

9. Le raycasting, explique pas a pas

C'est la partie la plus classique de soutenance cub3D.

9.1Idee de base

On imagine que le joueur est dans une grille 2D. Pour fabriquer l'image 3D, on n'envoie pas un seul rayon, mais un rayon par colonne de l'ecran.

Si l'ecran fait '1024' pixels de large, on lance '1024' rayons.

Pour chaque rayon:

1. on calcule sa direction
2. on avance de case en case dans la grille jusqu'a toucher un mur

3. on mesure la distance au mur
4. on convertit cette distance en hauteur de mur a dessiner
5. on choisit la bonne texture selon le cote du mur touche
6. on echantillonne la texture et on colorie la colonne

9.2 'cast_rays()'

'cast_rays()' boucle de 'x = 0' a 'x < largeur_ecran'.

Pour chaque colonne:

- 'init_ray()'
- 'step_dda()'
- 'compute_wall()'
- 'draw_column()'

Donc une colonne ecran correspond exactement a un rayon.

9.3 'init_ray()'

Cette fonction calcule tout ce qui est necessaire avant la marche DDA.

9.3.1 'camera_x'

Formule:

'camera_x = 2.0 * x / screen_width - 1.0'

Interet:

- colonne gauche -> proche de '-1'
- centre -> '0'
- colonne droite -> proche de '1'

On transforme donc un indice de pixel en position relative dans le champ de vision.

9.3.2 Angle du rayon

Formule du code:

'angle = player.angle + atan(camera_x * tan(PI / 6.0))'

Comme 'PI / 6 = 30 degres', le champ de vision total vaut environ '60 degres'.

Le centre de l'ecran regarde exactement dans 'player.angle'.

Les bords regardent un peu plus a gauche ou a droite.

9.3.3 Direction du rayon

On convertit l'angle en vecteur 2D:

- 'ray_dir_x = cos(angle)'
- 'ray_dir_y = sin(angle)'

9.3.4 Position de depart dans la grille

- 'map_x = (int)player.x'
- 'map_y = (int)player.y'

On recupere la case dans laquelle se trouve le joueur.

9.3.5 Distances DDA

- 'delta_x = abs(1 / ray_dir_x)'
- 'delta_y = abs(1 / ray_dir_y)'

Interpretation:

- 'delta_x' = distance a parcourir sur le rayon pour franchir une ligne verticale de grille
- 'delta_y' = distance a parcourir sur le rayon pour franchir une ligne horizontale de grille

Le petit '1e-9' evite une division exacte par zero.

9.3.6 Sens du pas

Si 'ray_dir_x < 0', le rayon va vers la gauche:

- 'step_x = -1'

Sinon il va vers la droite:

- 'step_x = 1'

Meme logique pour 'step_y'.

9.3.7 Premiere distance a une frontiere de case

'side_x' et 'side_y' mesurent la distance initiale avant le premier franchissement de ligne de grille.

C'est ce qui permet de savoir si le prochain pas DDA se fera:

- sur un bord vertical
- ou sur un bord horizontal

9.4 'step_dda()'

C'est la marche dans la grille.

Boucle:

- si 'side_x < side_y', on franchit d'abord une ligne verticale
- sinon, on franchit une ligne horizontale

Dans le premier cas:

- 'side_x += delta_x'
- 'map_x += step_x'
- 'side = 0'

Dans le second cas:

- 'side_y += delta_y'
- 'map_y += step_y'
- 'side = 1'

Puis on teste la case atteinte avec 'map_is_wall()'.

Dès qu'on touche un mur, on s'arrete.

Pourquoi cela marche?

Parce qu'au lieu d'avancer en continu avec de toutes petites valeurs, on saute directement d'intersection de grille en intersection de grille. C'est beaucoup plus efficace.

9.5 'compute_wall()'

Une fois le mur touche, il faut calculer la distance utile pour l'affichage.

9.5.1 Distance perpendiculaire

Formules:

- si 'side == 0':
'perp = (map_x - player.x + (1 - step_x) / 2.0) / ray_dir_x'
- sinon:
'perp = (map_y - player.y + (1 - step_y) / 2.0) / ray_dir_y'

On prend la distance perpendiculaire a la camera, pas la distance brute du rayon. Sinon on aurait l'effet "fish-eye": les murs sur les bords paraîtraient artificiellement plus lointains.

9.5.2 Hauteur de mur

Formule:

`'line_h = screen_height / perp'`

Interpretation:

- mur proche -> 'perp' petite -> 'line_h' grande
- mur lointain -> 'perp' grande -> 'line_h' petite

C'est exactement l'effet de perspective recherche.

9.5.3 Bornes verticales sur l'ecran

- `'start = -line_h / 2 + screen_height / 2'`
- `'end = line_h / 2 + screen_height / 2'`

Puis on clip pour rester dans la fenetre.

9.5.4 Choix de la texture selon l'orientation

Ici le code choisit entre quatre textures:

- `'side == 0'` et `'step_x > 0'` -> mur Est -> `'tex_ea'`
- `'side == 0'` et `'step_x < 0'` -> mur Ouest -> `'tex_we'`
- `'side == 1'` et `'step_y > 0'` -> mur Sud -> `'tex_so'`
- sinon -> mur Nord -> `'tex_no'`

C'est le coeur de la conformite du mandatory pour les quatre textures de murs.

9.6'draw_column()'

Cette fonction dessine la colonne de mur texturee.

9.6.1 Position exacte de l'impact sur le mur

On calcule d'abord `'wall_x'`, c'est-a-dire la coordonnee exacte du point d'impact sur le mur, puis on garde uniquement sa partie fractionnaire.

Cela donne une valeur entre `'0'` et `'1'` qui indique ou on a touche la face du mur.

9.6.2 Coordonne 'tex_x'

On la transforme en colonne de texture:

`'tex_x = wall_x * tex->w'`

Puis, selon le cote touche, on peut inverser `'tex_x'` pour eviter que la texture soit retournee horizontalement.

9.6.3 Parcours vertical de la texture

On veut etirer ou compresser la texture sur la hauteur de mur affichee.

- `'step = tex->h / line_h'`
- `'pos'` = position initiale dans la texture

Ensuite, pour chaque pixel `'y'` entre `'start'` et `'end'`:

- `'tex_y = (int)pos'`
- on lit le pixel couleur dans la texture
- on l'ecrit avec `'put_pixel()'`
- `'pos += step'`

9.7Pourquoi le rendu semble 3D alors qu'on dessine seulement des colonnes

Parce que:

- chaque colonne a sa propre distance
- les murs proches sont plus hauts
- les murs lointains sont plus bas

- le tout est calcule colonne par colonne avec un angle legerement different

Le cerveau reconstitue naturellement la profondeur.

10. Helpers utilitaires importants

10.1 'map_get()'

Cette fonction lit une case de la map sans risque.

Si la demande sort de la map, elle retourne un espace " ".

C'est une decision tres importante car cela permet de considerer automatiquement l'exterieur de la map comme du vide.

10.2 'map_is_wall()'

Cette fonction retourne '1' si la case vaut:

- "1"
- ou " "

Autrement dit, un espace est traite comme un mur/infranchissable. Cela sert a la fois:

- pour le raycasting
- pour la collision du joueur

10.3 'put_pixel()'

Fonction de base du rendu. Elle:

- verifie que l'image et les coordonnees sont valides
- calcule l'adresse memoire du pixel cible
- ecrit la couleur

Sans cette abstraction, le code de rendu serait beaucoup plus sale.

10.4 'str_len()', 'str_dup()', 'str_ndup()', 'str_chr()'

Ces utilitaires existent pour rester autonomes sans dependre d'une libft externe dans ce depot.

Ils servent au parsing et a GNL.

11. Nettoyage memoire et fermeture propre

11.1 'free_map_grid()'

Liberer:

- chaque ligne de 'map.grid'
- puis le tableau de pointeurs

Puis remettre:

- 'grid = NULL'
- 'height = 0'
- 'width = 0'

11.2 'free_cub()'

Liberer:

- les quatre chemins de textures
- la map

Cette fonction s'occupe du contenu logique de la scene.

11.3 'mlx_destroy()'

Liberer:

- la frame MLX
- les quatre textures chargees
- la fenetre
- le display/contexte MLX

Cette fonction s'occupe de la partie graphique.

11.4 'parse_fail()'

C'est le cleanup specialise du parsing.

Il nettoie tous les etats partiels qui peuvent exister si le programme n'a pas encore atteint la phase graphique.

11.5 'on_close()'

C'est la sortie finale en cas de fermeture utilisateur.

Chemin:

- 'mlx_destroy()'
- 'free_cub()'
- 'exit(0)'

12. Fonction par fonction, fichier par fichier

Cette section sert de pense-bete rapide avant la soutenance.

12.1 'src/main.c'

- 'dir_to_angle(char c)'
Convertit 'N/S/E/W' en angle de depart du joueur.
- 'setup_game(t_game *game)'
Transforme les donnees parsees en etat runtime: position reelle du joueur et couleurs compactees.
- 'main(int ac, char **av)'
Point d'entree du programme. Il orchestre tout le cycle de vie.

12.2 'src/init_struct.c'

- 'init_cub(t_cub *cub)'
Remet a zero toute la structure de scene logique.
- 'init_images(t_game *game)'
Remet a 'NULL' tous les pointeurs MLX et textures chargees.
- 'init_game_struct(t_game *game)'
Initialise tout l'etat runtime global, y compris le tableau des touches.

12.3 'src/parse_cub.c'

- 'init_gm(t_gm *gm, t_cub *cub)'

Initialise le contexte temporaire de parsing.

- 'parse_fail(t_gm *gm, char *msg)'
Nettoie un parsing partiellement construit et affiche l'erreur.
- 'check_path_map(char *path_map)'
Verifie extension '.cub' et non-dossier.
- 'parse_cub(t_cub *cub, char *path_map)'
Fonction centrale du parsing.

12.4 'src/init_read.c'

- 'trim_eol(char *line)'
Supprime fin de ligne Unix ou Windows.
- 'has_all_config(t_gm *gm)'
Verifie que les 6 configurations obligatoires ont ete vues.
- 'store_map_line(t_gm *gm)'
Ajoute une ligne de map et met a jour la largeur max.
- 'read_line(t_gm *gm)'
Classe une ligne: vide, config, ou contenu de map.
- 'read_cub_file(t_gm *gm)'
Lit tout le fichier ligne par ligne.

12.5 'src/good_map_utils.c'

- 'starts_with_id(char *s, char *id)'
Verifie un identifiant de config.
- 'gm_has_cub_ext(const char *s)'
Verifie l'extension '.cub'.
- 'gm_is_config_line(char *line)'
Verifie si une ligne ressemble a une config valide.
- 'gm_free_lines(char **lines, int count)'
Libere un tableau de lignes allouees.

12.6 'src/good_map_cfg.c'

- 'skip_spaces(char *s)'
Avance apres les espaces.
- 'trim_token(char *s)'
Duplique un token en retirant les espaces exterieurs.
- 'check_path(char *path)'
Verifie qu'un chemin existe et n'est pas un dossier.
- 'set_texture(char **dst, int *flag, char *s)'
Parse un chemin de texture, refuse les doublons et stocke le resultat.
- 'gm_parse_config(char *line, t_gm *gm)'
Parse une ligne de configuration en la dispatchant vers texture ou couleur.

12.7 'src/good_map_rgb.c'

- 'skip_spaces(char *s)'
Helper local pour ignorer les espaces.
- 'parse_value(char **s, int *value)'
Lit un entier decimal.

- 'parse_rgb(t_color *color, char *s)'
Parse 'R,G,B' complet avec validation de plage.
- 'gm_set_color(t_color *color, int *flag, char *s)'
Refuse doublon ou RGB invalide.

12.8 'src/good_map_push.c'

- 'gm_expand(char ***lines, int count, int *cap)'
Agrandit le tableau dynamique des lignes de map.
- 'gm_push_line(char ***lines, int *count, int *cap, char *line)'
Ajoute une ligne de map dupliquee dans le tableau.

12.9 'src/good_map_grid.c'

- 'cell_is_closed(char **grid, int y, int x)'
Verifie qu'une case jouable n'est pas collee a un espace.
- 'gm_is_closed(char **lines, int height, int width)'
Parcourt toute la map paddee pour verifier la fermeture.
- 'fill_row(char *row, int width)'
Remplit une ligne de padding avec des espaces.
- 'alloc_pad(int height, int width)'
Alloue la grille paddee.
- 'gm_pad_lines(char **lines, int height, int width)'
Copie la map au centre de la grille paddee.

12.10 'src/init_map.c'

- 'assign_map(t_gm *gm)'
Transfere la map construite depuis le contexte de parsing vers 'cub->map'.
- 'build_map(t_gm *gm)'
Valide la fermeture et le joueur, puis finalise la map.

12.11 'src/find_player.c'

- 'is_player(char c)'
Verifie si un caractere est une orientation joueur.
- 'find_player(t_map *map)'
Trouve la position/orientation initiale et impose qu'il y ait exactement un joueur.

12.12 'src/parsing.c'

- 'put_err(char *s)'
Ecrit un texte sur 'stderr'.
- 'not_directory(char *str)'
Verifie qu'un chemin n'est pas un dossier.
- 'print_error(char *msg)'
Affiche 'Error' puis un message explicite.
- 'str_chr(const char *s, int c)'
Equivalent minimal a 'strchr'.

12.13 'src/init_utils.c'

- 'is_map_char(char c)'
Verifie qu'un caractere est autorise dans la map.

- 'is_empty_line(char *line)'
Verifie qu'une ligne est vide ou pleine d'espaces.
- 'is_map_line(char *line)'
Verifie qu'une ligne est une ligne de map valide.
- 'str_len(const char *s)'
Equivalent minimal a 'strlen'.

12.14 'src/string_dup.c'

- 'str_ndup(const char *s, size_t len)'
Duplique 'len' caracteres.
- 'str_dup(const char *s)'
Duplique une chaine complete.

12.15 'src/mlx_init.c'

- 'load_texture(void *mlx, t_img *tex, char *path, char *name)'
Charge une texture XPM et recupere son buffer pixel.
- 'setup_mlx_core(t_game *g, int w, int h, char *title)'
Cree le contexte MLX, la fenetre et le framebuffer.
- 'destroy_images(t_game *g)'
Detruit toutes les images MLX allouees.
- 'mlx_setup(t_game *g, int w, int h, char *title)'
Initialise tout le sous-systeme graphique.
- 'mlx_destroy(t_game *g)'
Nettoie tout le sous-systeme graphique.

12.16 'src/input.c'

- 'map_key(int keycode)'
Traduit un keycode MLX en action interne.
- 'key_press(int keycode, t_game *g)'
Active une action de clavier.
- 'key_release(int keycode, t_game *g)'
Desactive une action de clavier.
- 'on_close(t_game *g)'
Ferme proprement le programme.

12.17 'src/map_get.c'

- 'map_get(t_map *m, int mx, int my)'
Lit une case de map avec gestion defensive des bords.
- 'map_is_wall(t_map *m, int mx, int my)'
Traite "1" et " " comme infranchissables.

12.18 'src/pixel.c'

- 'put_pixel(t_game *g, int x, int y, int color)'
Ecrit un pixel dans le framebuffer.

12.19 'src/render.c'

- 'can_move(t_game *g, double x, double y)'
Teste si le joueur peut occuper une position avec son collider.

- 'move_player(t_game *g, double fwd, double strafe)'
Applique les composantes de mouvement et gere le glissement sur les murs.
- 'update_player(t_game *g)'
Traduit l'etat des touches en mouvement/rotation.
- 'draw_floor_ceiling(t_game *g)'
Remplit le fond avec plafond et sol.
- 'render_frame(t_game *g)'
Dessine une frame complete.

12.20 'src/raycast.c'

- 'init_ray(t_game *g, t_ray *r, int x)'
Initialise un rayon pour une colonne d'ecran.
- 'step_dda(t_game *g, t_ray *r)'
Fait progresser le rayon de case en case jusqu'au mur.
- 'compute_wall(t_game *g, t_ray *r)'
Calcule distance, hauteur, bornes d'affichage et choisit la texture.
- 'draw_column(t_game *g, t_ray *r, t_img *tex, int x)'
Dessine la portion de mur texturee sur la colonne 'x'.
- 'cast_rays(t_game *g)'
Lance le raycasting complet sur toute la largeur de la fenetre.

12.21 'src/cleanup.c'

- 'free_map_grid(t_map *map)'
Libere la grille de map.
- 'free_cub(t_cub *cub)'
Libere la scene logique complete.

12.22 'src/good_map.c'

- 'good_map(char *path_map)'
Helper de validation simple: parse puis free. Cette fonction n'est pas au coeur du runtime principal, mais elle peut servir pour des tests rapides.

12.23 'get_next_line/get_next_line.c'

- 'gnl_buffer(void)'
Donne acces au buffer statique interne.
- 'free_if_err(char **buffer, int *rd)'
Nettoie le buffer si 'read()' a echoue.
- 'get_next_line(int fd)'
Lit et retourne une ligne complete.
- 'gnl_clear(void)'
Vide explicitement le buffer statique.

12.24 'get_next_line/get_next_line_utils.c'

- 'gnl_strchr(char *s, char c)'
Cherche un caractere dans une chaine.
- 'gnl_strdup(const char *s, int ln)'
Duplique une chaine, avec option d'ajout de '
'.

- 'gnl_strjoin(char *s1, char *s2)'
Concatene deux chaines.
- 'read_it(int fd, char **buffer, int *reader)'
Lit un bloc et l'ajoute au buffer statique.
- 'extract_line(char **buffer)'
Extrait la prochaine ligne complete du buffer.

13. Ce qu'il faut savoir expliquer a la soutenance

13.1 Pourquoi le joueur est place avec '+ 0.5'

Parce que la map est une grille de cases. Si on laissait le joueur au coin haut-gauche de sa case, les collisions et les rayons seraient faux. Le centre de la case est la position logique correcte.

13.2 Pourquoi les espaces sont traites comme des murs

Parce qu'un espace represente du vide hors zone jouable. Si on ne le traitait pas comme un mur:

- le joueur pourrait sortir de la map
- le raycasting pourrait partir dans le vide
- une map apparemment fermee pourrait en fait contenir une fuite par ses espaces

13.3 Pourquoi on utilise la distance perpendiculaire et non la distance brute du rayon

Pour supprimer l'effet fish-eye. C'est une question classique d'evaluation.

13.4 Pourquoi on dessine d'abord sol/plafond puis les murs

Parce que les murs ne recouvrent qu'une partie verticale de l'ecran. Le plus simple est:

1. peindre tout le fond
2. venir ecraser la zone murale texturée par-dessus

13.5 Pourquoi le mouvement est gere par etat de touches et non directement dans 'key_press'

Parce que cela donne un mouvement continu et fluide. Si on deplacait le joueur uniquement lors d'un evenement ponctuel, le jeu serait saccade.

13.6 Pourquoi le parsing est separe du runtime

Parce qu'un parseur doit construire une scene valide avant meme que le moteur graphique commence. Cela simplifie:

- les erreurs
- la memoire
- les tests
- la lisibilite du code

14. Conformite au sujet mandatory

Sur la base du code actuel:

- MiniLibX: oui

- fenetre et boucle de rendu: oui
- quatre textures murales 'N/S/E/W': oui
- deux couleurs differentes sol/plafond: oui
- fleche gauche/droite: oui
- 'W/A/S/D': oui
- 'ESC': oui
- croix rouge: oui
- argument '.cub': oui
- map fermee: oui
- un seul joueur: oui
- messages d'erreur explicites: oui
- README a la racine du repo: oui
- bonus: non, volontairement

15. Conclusion a retenir pour l'oral

La meilleure facon de presenter ce projet est la suivante:

1. expliquer qu'on parse d'abord une scene complete et valide
2. montrer que le runtime convertit cette scene en etat de jeu
3. expliquer que chaque frame fait: input -> mouvement -> fond -> raycasting -> affichage
4. insister sur le raycasting: un rayon par colonne, DDA, distance perpendiculaire, hauteur projete, texture selon orientation
5. finir par la gestion propre des erreurs et de la memoire

Si vous savez raconter ce fil sans hesiter, vous maitrisez l'essentiel du projet.